

Perplexity, Worked Out by Hand

The standard yardstick for a language model’s quality — the exponentiated average negative log-likelihood, read as the model’s effective “number of choices” per token — computed on a five-word sentence.

What this document does. It takes a short sentence, lists the probability the model assigned to each *actual* next word, and walks the full chain: per-token log-probability → average cross-entropy → exponentiate → perplexity. It then interprets the number as a branching factor, compares a good model against a bad one on the same sentence, and pins down the uniform-distribution baseline. Numbers from Appendix A.

Prerequisite: softmax (#1) supplies the per-token probabilities; perplexity aggregates them across a sequence.

1 From one token to a whole sentence

A language model is a probability machine: at each position it outputs $P(\text{next token} \mid \text{context})$. To score the model on a real sentence $w_1, \dots, w_N$ we ask how much probability it placed on the words that *actually* came next. By the chain rule the sentence probability is the product

$$P(w_1, \dots, w_N) = \prod_{t=1}^N P(w_t \mid w_{<t}).$$

Products of many small numbers underflow and are hard to compare across lengths, so we work in the log domain and average per token — that average is the *cross-entropy*, and its exponential is *perplexity*.

Intuition

Perplexity answers: “*on average, how many equally-likely words was the model choosing between?*” A perplexity of 1 means total certainty (it always put 100% on the right word). A perplexity of 5 on a 5-word vocabulary means the model learned nothing — it was as confused as a fair die. Lower is better: it is a count of effective options, and fewer options means a more confident, better-fit model.

2 The formula

For a sequence of N tokens with model probabilities $p_t = P(w_t \mid w_{<t})$,

$$\text{PPL} = \exp\left(\underbrace{-\frac{1}{N} \sum_{t=1}^N \ln p_t}_{\text{cross-entropy } H}\right) = \left(\prod_{t=1}^N p_t\right)^{-1/N}$$

The two forms are identical: exp of an average of logs is the *geometric mean*. So perplexity is exactly the reciprocal of the geometric-mean per-token probability — the typical probability the model gives the truth, inverted into a count.

3 Worked by hand on a five-word sentence

Sentence "the cat sat on mat". The model assigned these probabilities to each correct next word:

t	1	2	3	4	5
w_t	the	cat	sat	on	mat
p_t	0.40	0.20	0.50	0.80	0.10
$\ln p_t$	-0.9163	-1.6094	-0.6931	-0.2231	-2.3026

Step 1 — sum the log-probabilities.

$$\sum_t \ln p_t = -0.9163 - 1.6094 - 0.6931 - 0.2231 - 2.3026 = -5.7446.$$

Step 2 — average (negate) to get cross-entropy.

$$H = -\frac{1}{5}(-5.7446) = 1.1489 \text{ nats/token.}$$

Step 3 — exponentiate.

$$\text{PPL} = e^{1.1489} = 3.1548.$$

The model was, on average, as uncertain as if choosing uniformly among ≈ 3.15 **words** at each step. Equivalently, the geometric-mean probability it gave the correct word was $1/3.1548 = 0.317$.

Mini-example • one confident token rescues the average

Position 4 (on, $p = 0.80$) contributes only $-\ln 0.80 = 0.223$ to the sum, while position 5 (mat, $p = 0.10$) contributes $-\ln 0.10 = 2.303$ — over *ten times* the penalty. Because perplexity is a geometric mean, a single badly predicted token (a small p_t) drags the whole score up far more than a confident token pulls it down. Tail mistakes dominate perplexity.

4 Reading the number: better vs. worse models

Perplexity is only meaningful in comparison. On the *same* sentence:

model	per-token p_t	PPL
our model	(0.40, 0.20, 0.50, 0.80, 0.10)	3.155
stronger model	(0.60, 0.50, 0.70, 0.90, 0.40)	1.676
uniform guess ($V=5$)	(0.2, 0.2, 0.2, 0.2, 0.2)	5.000

The stronger model assigns more mass to the truth at every step, so its perplexity falls toward 1. The uniform model — which has learned nothing — sits exactly at the vocabulary size $V = 5$, the worst non-degenerate score. **Any useful model must beat V ; state-of-the-art LLMs reach single-digit perplexity over vocabularies of 50,000+.**

Intuition

Two caveats that bite in practice. (1) Perplexity is only comparable across models that share the *same tokenizer and vocabulary* — change how text is split and the per-token probabilities, and thus PPL, shift. (2) Low perplexity means the model predicts text well, not that its outputs are *useful, truthful, or safe*; it is a fluency gauge, not a quality verdict. It is the cheap, automatic metric — pair it with task evals (Track 6), never replace them.

Equation cheat-sheet

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_t \ln p_t\right) = \left(\prod_t p_t\right)^{-1/N} = \frac{1}{\text{geometric-mean } p_t}$$

$$\text{PPL} = 1 \Rightarrow \text{perfect}, \quad \text{PPL} = V \Rightarrow \text{uniform / no learning}$$

lower is better; comparable only across a shared tokenizer/vocabulary

Appendix A · Reference implementation

```
import numpy as np
```

```
def perplexity(p):                                # p = probs of the actual next tokens
    p = np.asarray(p, float)
    return float(np.exp(-np.log(p).mean()))
```

```
p = [0.40, 0.20, 0.50, 0.80, 0.10]
```

```
perplexity(p)                                     # 3.1548
```

```
perplexity([0.60, 0.50, 0.70, 0.90, 0.40])      # 1.6761 (stronger model)
```

```
perplexity([0.2]*5)                              # 5.0      (uniform over V=5)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0